SECAO C — FUNDAMENTACAO MATEMATICA DA CAMADA DE ALGORITMOS

TrueCompass · Documentacao Tecnica · Secao complementar as secoes 2 a 6 · Notacao em ASCII por restricao de fonte

Esta secao formaliza os resultados matematicos que sustentam a implementacao da camada de algoritmos, com enunciado e demonstracao de cada um, e registra as medicoes experimentais obtidas. Complementa e, em um ponto, **retifica** a analise da secao 6. Cada resultado indica o arquivo que o implementa.

C.1 METRICA SOBRE A SUPERFICIE TERRESTRE

Definicao C.1.

Dados dois pontos de coordenadas geograficas (lat1, lon1) e (lat2, lon2), a distancia de grande circulo entre eles e dada pela formula de haversine:

```
a = sen^2((lat2 - lat1)/2) + cos(lat1) * cos(lat2) * sen^2((lon2 - lon1)/2)
d = 2 * R * arcsen(raiz(a)), com R = 6.371.000 m
```

Motivacao. A adocao da metrica esferica, em lugar da distancia euclidiana sobre as coordenadas em graus, decorre da anisotropia do sistema de coordenadas geograficas: os meridianos convergem nos polos, de modo que a extensao linear de um grau de longitude depende da latitude. Na latitude de -23,5 graus (Sao Paulo), medem-se:

Eixo	Extensao de 1 grau	Observacao
Latitude	111,2 km	praticamente invariante
Longitude	102,0 km	reduzida pelo fator cos(latitude)

O emprego da distancia euclidiana sobre graus incorreria, portanto, em erro de **8,3%** no eixo leste-oeste nessa latitude — equivalente a 250 m em um raio de busca de 3 km, magnitude suficiente para alterar a selecao do motorista. O termo cos(lat1) * cos(lat2) na Definicao C.1 e precisamente a correcao dessa anisotropia.

Proposicao C.1.

A funcao d da Definicao C.1 e uma metrica sobre a esfera; em particular, satisfaz a desigualdade triangular d(x, z) <= d(x, y) + d(y, z).

> **Demonstracao.** d e a distancia geodesica na esfera, isto e, o comprimento do menor arco de grande circulo unindo os pontos. Distancias geodesicas em variedades riemannianas conexas satisfazem os axiomas de metrica. A desigualdade triangular sera invocada na Proposicao C.5 e na Proposicao C.6. **|**
>
> *Implementacao: algoritmos/grafo.py. A funcao e invocada em quatro contextos distintos — peso de aresta, heuristica do A*, poda da quadtree e ordenacao por proximidade —, sendo a rotina mais executada do sistema.*

C.2 FUNCAO PESO E ANALISE DIMENSIONAL

Definicao C.2.

Para uma aresta (u, v) representando um trecho de via de velocidade declarada V, em km/h, o peso e definido por:

```
w(u, v) = d(u, v) / (V / 3,6)

Analise dimensional: [m] / ([m] / [s]) = [s]
```

O fator 3,6 converte km/h em m/s. Segue que **w assume valores em segundos, e nao em metros**: a funcao peso mede **tempo de deslocamento**. Esta e a decisao de modelagem de maior consequencia em todo o sistema, uma vez que o objetivo operacional e minimizar o tempo de espera do passageiro, e nao a distancia percorrida. Uma via arterial de 50 km/h pode, portanto, ser preferida a uma via local de 30 km/h ainda que geometricamente mais longa.

> *Consequencia para manutencao: alterar a unidade de w para metros invalidaria simultaneamente a admissibilidade da heuristica (Proposicao C.5), o calculo tarifario (Definicao C.8) e a ordenacao do pareamento, sem gerar erro de execucao. A suite de testes detecta a alteracao.*

C.3 REPRESENTACAO DO GRAFO E ESPARSIDADE

Proposicao C.2.

A malha viaria e um grafo esparso, e sua representacao por lista de adjacencia ocupa espaco O(V + E), contra O(V^2) da matriz de adjacencia.

Demonstracao. Cada vertice representa um cruzamento, cujo grau e limitado pelo numero de vias incidentes — grandeza que nao cresce com o tamanho da cidade. Logo E = O(V), e a lista de adjacencia ocupa O(V + E) = O(V). A matriz reserva espaco para as V^2 ligacoes possiveis, das quais a fracao efetivamente utilizada e O(1/V). **|**

Medicao sobre a malha de referencia do projeto:

Grandeza	Valor medido
Vertices (V)	3.600
Arestas (E)	8.545
Grau medio (E/V)	**2,37**
Entradas — lista de adjacencia	aprox. 12.145
Entradas — matriz de adjacencia	12.960.000 (**1.067x** superior)

C.4 CORRETUDE E COMPLEXIDADE DO ALGORITMO DE DIJKSTRA

Proposicao C.3.

(Invariante de extracao.) Em um grafo com w(e) >= 0 para toda aresta e, no instante em que um vertice u e extraido da fila de prioridade, dist[u] e igual ao custo do caminho minimo de s a u.

Demonstracao. Suponha, por absurdo, que exista caminho de s a u de custo inferior a dist[u]. Tal caminho contem necessariamente algum vertice x ainda presente na fila, e seu custo e dist[x] + c, onde c e o custo do trecho de x a u. Como u foi extraido por ser o minimo da fila, vale dist[x] >= dist[u]; e como todos os pesos sao nao negativos, vale c >= 0. Logo dist[x] + c >= dist[u], contradizendo a hipotese. A prova por inducao no numero de extracoes consta da secao 5.1. **|**

Corolario C.1.

A hipotese de nao negatividade dos pesos e indispensavel: admitido w(e) < 0, o trecho c poderia reduzir o custo total e o invariante falharia.

Protecao em codigo: `Grafo.adicionar_aresta` rejeita peso negativo com mensagem que remete a esta secao. A unica hipotese da prova e verificada em tempo de construcao do grafo, e nao apenas documentada.

Corolario C.2.

E licito encerrar a busca no instante em que o vertice de destino e extraido da fila, dispensando o calculo das distancias aos demais vertices.

Proposicao C.4.

Com fila de prioridade implementada como heap binario, a complexidade e O((V + E) log V).

Demonstracao. Cada vertice e extraido exatamente uma vez, a custo O(log V), totalizando O(V log V). Cada aresta origina no maximo uma insercao, a custo O(log V), totalizando O(E log V). **|**

Nota de implementacao: a biblioteca padrao de Python nao oferece a operacao DECREASE_KEY. Adota-se remocao preguicosa (lazy deletion) — entradas obsoletas sao inseridas e descartadas na extracao. O efeito assintotico e equivalente.

C.5 ALGORITMO A*: CONDICOES SOBRE A HEURISTICA

Definicao C.3.

O A* ordena a fila de prioridade pela funcao de avaliacao f(v) = g(v) + h(v), em que g(v) e o custo acumulado de s a v e h(v) uma estimativa do custo remanescente de v ao destino t.

Definicao C.4.

h e **admissivel** quando h(v) <= h*(v) para todo v, sendo h*(v) o custo real do caminho minimo de v a t.

Definicao C.5.

h e **consistente** quando h(u) <= w(u, v) + h(v) para toda aresta (u, v) — desigualdade triangular sobre a heuristica.

Definicao C.6.

A heuristica adotada no projeto e:

```
h(v) = d(v, t) / V_max, com V_max = velocidade maxima da malha
```

Proposicao C.5.

A heuristica da Definicao C.6 e admissivel.

> **Demonstracao.** Seja v = x0, x1, ..., xk = t o caminho minimo de v a t. Seu custo e h*(v) = SOMA d(xi, xi+1) / Vi, onde Vi <= V_max e a velocidade do trecho i. Como Vi <= V_max, vale d(xi, xi+1)/Vi >= d(xi, xi+1)/V_max, logo h*(v) >= (1/V_max) * SOMA d(xi, xi+1). Pela desigualdade triangular (Proposicao C.1), SOMA d(xi, xi+1) >= d(v, t). Portanto h*(v) >= d(v, t)/V_max = h(v). |

Proposicao C.6.

A heuristica da Definicao C.6 e consistente.

> **Demonstracao.** Pela Proposicao C.1, d(u, t) <= d(u, v) + d(v, t). Dividindo por V_max: h(u) <= d(u, v)/V_max + h(v). Pela Definicao C.2, w(u, v) = d(u, v)/V(u,v) com V(u,v) <= V_max, donde w(u, v) >= d(u, v)/V_max. Combinando: h(u) <= w(u, v) + h(v). |

As duas propriedades produzem garantias distintas:

Propriedade	Garantia	Consequencia de sua ausencia
Admissibilidade	solucao otima	o algoritmo devolve rota de custo superior ao minimo
Consistencia	nenhum vertice reexpandido	reexpansoes degradam o desempenho, sem perda de otimalidade

Corolario C.3.

O algoritmo de Dijkstra e o caso particular do A* com h identicamente nula, que e trivialmente admissivel e consistente. Ambos sao, na implementacao, a mesma rotina com parametro distinto.

Sobre a complexidade. O A* preserva a complexidade de pior caso O((V + E) log V) da Proposicao C.4; o ganho manifesta-se no caso medio, pela reducao do numero de vertices expandidos. A metrica pertinente para o relatorio experimental e, por conseguinte, a **contagem de vertices expandidos**, e nao a notacao assintotica.

> *Erro de implementacao frequente: omitir a divisao por V_max faz h retornar metros enquanto w e expresso em segundos. A estimativa supera a realidade por duas ordens de grandeza, a admissibilidade e perdida e as rotas deixam de ser otimas, sem qualquer erro de execucao. O teste `test_heuristica_inadmissivel_seria_pega` reproduz deliberadamente essa falha.*

C.6 DUALIDADE DO GRAFO REVERSO E PAREAMENTO MANY-TO-ONE

Definicao C.7.

O grafo reverso de G = (V, E, w) e G^R = (V, E^R, w^R), em que (v, u) pertence a E^R com w^R(v, u) = w(u, v) se e somente se (u, v) pertence a E.

Proposicao C.7.

(Dualidade.) Para quaisquer p, m em V vale dist_{G^R}(p, m) = dist_G(m, p).

> **Demonstracao.** Estabelece-se bijecao entre os conjuntos de caminhos. A todo caminho P = (m = v0, v1, ..., vk = p) em G associa-se P^R = (p = vk, ..., v0 = m) em G^R. A associacao esta bem definida, pois cada aresta (vi, vi+1) de E corresponde a (vi+1, vi) em E^R; e preserva o custo, pois w^R(vi+1, vi) = w(vi, vi+1) por construcao, e a soma nao depende da ordem das parcelas. Sendo a correspondencia bijetiva e preservadora de custo, os minimos dos dois conjuntos coincidem. |

Corolario C.4.

O problema many-to-one — determinar dist_G(mi, p) para k origens mi e um unico destino p — reduz-se a uma unica execucao one-to-many sobre G^R a partir de p.

O custo do pareamento decresce de k * O((V + E) log V) para uma unica busca, limitada por raio e encerrada quando os k alvos houverem sido extraidos (Corolario C.2). A arvore de caminhos minimos resultante fornece, adicionalmente, as k rotas sem custo suplementar: percorrer os ponteiros de predecessor a partir de mi, na ordem natural, produz a rota mi -> p em G.

> *A inversao e indispensavel por ser G direcionado (vias de sentido unico). Executar a busca a partir de p sobre G, e nao sobre G^R, responderia ao problema distinto de determinar dist_G(p, mi).*

C.7 COMPONENTE FORTEMENTE CONEXO

Proposicao C.8.

Seja R+(s) o conjunto dos vertices alcancaveis a partir de s em G, e R-(s) o conjunto dos alcancaveis a partir de s em G^R. Entao o componente fortemente conexo que contem s e CFC(s) = R+(s) INTERSECAO R-(s).

> **Demonstracao.** Por definicao, v pertence a CFC(s) se e somente se s alcanca v e v alcanca s em G. A primeira condicao equivale a v pertencer a R+(s). A segunda, pela Proposicao C.7, equivale a v pertencer a R-(s). A equivalencia das duas condicoes conjuntamente com a pertinencia a intersecao conclui. |

Aplicacao. Extratos do OpenStreetMap delimitados por caixa envolvente truncam vias na fronteira, produzindo componentes isolados e vertices de sentido unico sem aresta de saida. Um motorista posicionado em tal vertice seria selecionado pelo pareamento sem que exista rota ate o passageiro. A poda por CFC elimina a ocorrencia por construcao, e reutiliza o grafo reverso ja instanciado para o pareamento — trata-se do passo central do algoritmo de Kosaraju.

> *Ressalva: o procedimento devolve o componente que contem a semente, e nao o de maior cardinalidade. Adotando-se como semente o vertice de maior grau, o componente obtido coincide, em malhas viarias, com o componente gigante. Medicao no conjunto de teste: 42,9% dos vertices descartados.*

C.8 ORDENACAO TOPOLOGICA E GARANTIA ESTRUTURAL DO FLUXO

Definicao C.8.

O grau de entrada de v e a cardinalidade do conjunto {u : (u, v) pertence a E}.

Proposicao C.9.

O algoritmo de Kahn produz ordenacao topologica valida de todo DAG, e detecta a presenca de ciclo: se ao termino a ordem obtida tem cardinalidade inferior a |V|, o grafo contem ciclo.

> **Demonstracao.** A validade da ordenacao e demonstrada por inducao em |V| na secao 5.2. Para a deteccao: se o grafo e aciclico, possui vertice de grau de entrada nulo, e a remocao iterada preserva a aciclicidade, de modo que todos os vertices sao processados. Reciprocamente, vertices pertencentes a um ciclo conservam grau de entrada positivo ao longo de toda a execucao, jamais ingressam na fila, e constituem exatamente o deficit observado. |

Proposicao C.10.

A complexidade do algoritmo de Kahn e O(V + E).

> **Demonstracao.** Cada vertice ingressa na fila no maximo uma vez; cada aresta e percorrida exatamente uma vez, no decremento do grau de entrada do sucessor. |

Aplicacao. O ciclo de vida da corrida e declarado como DAG, e a Proposicao C.9 e verificada na inicializacao do servidor: detectado ciclo, a aplicacao **nao entra em operacao**. A demonstracao da secao 5.2 converte-se, assim, em garantia de execucao.

Proposicao C.11.

No DAG do ciclo da corrida, o estado `paga` possui grau de entrada igual a 1, com unico predecessor `concluida`. Consequentemente, nenhuma sequencia de transicoes admissiveis conduz a `paga` sem passagem previa por `concluida`.

> **Demonstracao.** Imediata da estrutura do grafo: sendo `concluida` o unico predecessor de `paga`, todo caminho que termina em `paga` contem `concluida` em posicao imediatamente anterior. |

> *A Proposicao C.11 constitui a formalizacao do requisito funcional segundo o qual o pagamento e processado somente apos a conclusao da corrida. O requisito e assegurado pela **topologia do grafo**, e nao por verificacao condicional dispersa no codigo: sua alteracao exige alterar o grafo. A tabela `corrida_transicoes` registra cada aresta percorrida, constituindo evidencia empirica de conformidade.*

C.9 QUADTREE: RECORRENCIA, PODA E RETIFICACAO DA SECAO 6

Proposicao C.12.

A descida da raiz a uma folha de quadtree sobre n pontos uniformemente distribuidos obedece a recorrencia T(n) = T(n/4) + O(1), cuja solucao e Theta(log n).

> **Demonstracao.** Aplica-se o Teorema Mestre a T(n) = a * T(n/b) + f(n) com a = 1, b = 4 e f(n) = O(1). Tem-se n^(log_b a) = n^(log_4 1) = n^0 = 1, donde f(n) = Theta(n^(log_b a)). Recai-se no caso 2, cuja solucao e T(n) = Theta(n^(log_b a) * log n) = Theta(log n). |

Proposicao C.13.

(Limite inferior que autoriza a poda.) Seja Q = [lat_min, lat_max] x [lon_min, lon_max] e (lat, lon) o ponto de consulta. Definindo lat* = min(max(lat, lat_min), lat_max) e analogamente lon*, o valor d((lat, lon), (lat*, lon*)) e limite inferior da

distancia de (lat, lon) a qualquer ponto de Q.

> **Demonstracao.** O ponto (lat*, lon*) resulta da projecao independente de cada coordenada sobre o intervalo correspondente, minimizando simultaneamente |lat - y1| e |lon - y2| sobre (y1, y2) em Q. Como d e monotona nao decrescente em cada uma dessas diferencas, com a outra fixada, nenhum ponto de Q dista menos. **|**

Corolario C.5.

Excedendo o limite inferior o raio de consulta, o subarvore correspondente pode ser descartada integralmente, sem inspecao de seus pontos.

> *A monotonicidade invocada na Proposicao C.13 e exata quanto a longitude e aproximada quanto a latitude, dada a variacao de cos(latitude) no interior de Q. Adota-se, por isso, folga de 1% sobre o raio ao podar (constante FOLGA_PODA): a inspecao de subarvore supranumeraria e admissivel; o descarte de subarvore contendo resultado, nao.*

Retificacao da secao 6. A secao 6 atribui complexidade O(log n) a consulta por raio. A formulacao correta e:

```
Consulta por raio: O(log n + k), k = numero de resultados
```

Proposicao C.14.

A consulta por raio em quadtree tem complexidade O(log n + k), com k a cardinalidade do conjunto-resposta. O termo k e irredutivel.

> **Demonstracao.** O termo log n corresponde a descida (Proposicao C.12). O termo k decorre da necessidade de enumerar os pontos-resposta: qualquer estrutura que os devolva deve lê-los, o que estabelece Omega(k) como limite inferior da operacao, independentemente da indexacao adotada. **|**

Medicao experimental — consulta em raio de 300 m, 200 consultas por tamanho de conjunto:

Pontos (n)	Varredura linear (comparacoes)	Quadtree (nos visitados)	Razao	Altura
500	500	21,2	23,6x	5
2.000	2.000	41,6	48,1x	6
8.000	8.000	85,0	94,1x	8
32.000	32.000	206,7	**154,8x**	9

O crescimento observado — aumento de 64 vezes em n correspondendo a aproximadamente 10 vezes em nos visitados — e compativel com O(log n + k): a raio constante, o adensamento do conjunto eleva k, que passou de aproximadamente 5 para 320. **A vantagem assintotica da quadtree reside em que a varredura linear incorre em custo n mesmo quando k = 0**; consulta sobre regiao desprovida de pontos encerra-se na raiz, com um unico no visitado.

C.10 FUNCAO TARIFARIA

Definicao C.9.

O custo estimado da corrida e a funcao afim do tempo e da distancia:

```
custo = 5,00 + 0,80 * (segundos / 60) + 1,90 * (metros / 1000)
```

A funcao e monotona crescente em ambas as variaveis. Distingue-se das demais formulacoes desta secao por ser a unica cujos coeficientes nao derivam de resultado matematico, constituindo parametro de negocio livremente ajustavel.

C.11 CONSIDERACOES DE ARITMETICA DE PONTO FLUTUANTE

A adicao em ponto flutuante nao e associativa. O pareamento ingenuo (k buscas sobre G) e o pareamento reverso (busca unica sobre G^R) somam o **mesmo** conjunto de pesos em **ordens distintas**, conforme estabelecido na Proposicao C.7. Registrou-se divergencia maxima de **1,1 x 10^-13 s** sobre valores da ordem de 350 s — cerca de 3 x 10^-16 em termos relativos, compativel com o epsilon da precisao dupla.

Consequencia metodologica. A verificacao de equivalencia entre implementacoes emprega tolerancia relativa, e nao igualdade exata. A divergencia observada e ruido de arredondamento, e nao discrepancia algoritmica; a distincao entre ambas as hipoteses foi estabelecida por inspecao da magnitude.

C.12 VALIDACAO POR ORACULO

Adota-se como metodo de verificacao a **validacao por oraculo**: para cada rotina otimizada mantem-se em producao a implementacao ingenua correspondente, de corretude manifesta, e verifica-se a identidade dos resultados sobre entradas aleatorias. As implementacoes ingenuas nao constituem codigo inativo: desempenham a funcao de gabarito.

Oraculo	Rotina validada	Protocolo de verificacao
Dijkstra (h = 0)	A*	200 pares origem-destino aleatorios; custos identicos a menos de tolerancia relativa de 1e-9
Varredura linear	Quadtree	200 consultas aleatorias; identidade do conjunto-resposta e da ordenacao
Pareamento ingenuo	Pareamento reverso	identidade dos tempos estimados de chegada

A suite compreende 103 verificacoes, executaveis sem dependencia de framework de teste, de banco de dados ou de acesso a rede.

Sintese das medicoes de desempenho:

Substituicao	Metrica	Ganho	Otimalidade
k Dijkstras -> 1 busca reversa	nos expandidos	**10,7x**	preservada
Dijkstra -> A*	nos expandidos	**3,2x**	preservada
Varredura linear -> quadtree	nos visitados	**154,8x**	preservada

Registra-se que em nenhum dos tres casos houve perda de otimalidade: as rotinas otimizadas devolvem a **mesma** solucao, obtida com menor exploracao do espaco de busca.

C.13 EXTENSAO PREVISTA: EMPARELHAMENTO DE CUSTO MINIMO

A politica vigente de alocacao — atribuir a cada solicitacao o motorista de menor tempo estimado de chegada — e gulosa, e nao otimiza o resultado agregado quando multiplas solicitacoes concorrem pelo mesmo conjunto de motoristas. A formulacao adequada e o **emparelhamento bipartido de custo minimo** entre o conjunto de passageiros e o de motoristas, resoluvel pelo algoritmo hungaro em O(n^3).

A Proposicao C.7 permanece central nessa extensao: a construcao da matriz de custos demanda uma busca reversa **por passageiro**, e nao uma busca por par passageiro-motorista. Trata-se da extensao natural do projeto, e introduz na fundamentacao o tratamento matricial ausente na formulacao atual.